

Transactions: ACID Properties & Isolation Levels

1. What is a Transaction?

A **transaction** is a logical unit of database work that should be treated as one operation.

For example, suppose a payment system needs to:

1. Create a payment record.
2. Update the invoice as paid.
3. Store payment history.

These operations should not leave the database in a half-completed state.

```
BEGIN TRANSACTION

Insert payment
Update invoice
Insert payment history

COMMIT
```

If something fails:

```
ROLLBACK
```

So the basic idea is:

Either all required database changes happen, or none of them do.

2. ACID Properties

ACID stands for:

Property	Meaning
Atomicity	All operations succeed or all are rolled back
Consistency	Transaction moves DB from one valid state to another valid state
Isolation	Concurrent transactions should not incorrectly interfere with each other
Durability	Committed data survives failures

A — Atomicity

Atomicity means a transaction is **all-or-nothing**.

Example

Suppose we transfer ₹1,000:

```
Account A: -1000
Account B: +1000
```

Imagine:

```
Debit A    ✓
Credit B   ✗
```

Without atomicity, ₹1,000 disappears.

With a transaction:

```
BEGIN;

UPDATE account
SET balance = balance - 1000
WHERE id = 1;

UPDATE account
SET balance = balance + 1000
WHERE id = 2;

COMMIT;
```

If the second statement fails:

```
ROLLBACK
```

The first update is also undone.

Interview answer

Atomicity ensures that all operations within a transaction are completed successfully as a unit; otherwise the entire transaction is rolled back.

3. C — Consistency

Consistency means the database must obey its **defined rules and constraints** before and after the transaction.

For example:

```
Account balance cannot be negative
Payment amount > 0
Foreign key must reference an existing record
Unique constraint cannot be violated
```

Suppose:

```
balance >= 0
```

is a business/database constraint.

A transaction should not take the database from:

```
Valid state
  ↓
Invalid state
```

It should go:

```
Valid state
  ↓
Valid state
```

Important interview nuance

Consistency is **not simply “the data is correct.”**

It is about preserving:

- Primary-key constraints
- Foreign-key constraints
- Unique constraints
- Check constraints
- Application/business rules

Some business rules exist only in application code, while others are enforced directly by the database.

Interview answer

Consistency ensures that a transaction preserves database integrity constraints and business invariants, taking the database from one valid state to another valid state.

4. I — Isolation

This is the most important part of today's topic.

Imagine two transactions executing simultaneously:

```
Transaction A
Transaction B
```

Without appropriate isolation, A may see intermediate or changing data from B.

Isolation controls **how much one transaction can see from another concurrent transaction.**

For example:

```
A: Read balance = 1000
```

B: Update balance = 500

A: Read balance again

Should A see:

1000

or

500

That depends on the isolation level.

5. D — Durability

Once a transaction is committed:

```
COMMIT;
```

the database promises that the committed result will survive normal failures such as:

- process crash
- database restart
- power failure

Databases achieve this using mechanisms such as **transaction logs / WAL / redo logging**, depending on the database.

Conceptually:

```
COMMIT
↓
Data is persisted
↓
DB crashes
↓
DB restarts
↓
Committed transaction remains
```

Interview answer

Durability ensures that once a transaction is committed, its changes persist even if the system subsequently crashes.

6. Isolation Levels

The standard isolation levels are:

```
READ UNCOMMITTED
READ COMMITTED
REPEATABLE READ
SERIALIZABLE
```

There is a trade-off:

```
More concurrency
    ↓
Less isolation

Less concurrency
    ↓
More isolation
```

Higher isolation generally means stronger correctness guarantees but potentially more locking/contention and lower concurrency.

7. Database Concurrency Problems

Before understanding isolation levels, understand these anomalies.

7.1 Dirty Read

Transaction A reads data that Transaction B has changed but **not committed**.

Example:

```
Initial balance = 1000

A: UPDATE balance = 500
   (not committed)

B: READ balance
   → sees 500

A: ROLLBACK
```

B read data that never actually became committed.

That's a:

Dirty Read

8. Non-Repeatable Read

Transaction A reads the same row twice and gets different values because another transaction committed an update between the reads.

```
Initial balance = 1000

A: SELECT balance
   → 1000
```

```
B: UPDATE balance = 500
B: COMMIT

A: SELECT balance again
  → 500
```

Same query, same transaction, different result.

That's a:

Non-repeatable read

9. Phantom Read

A transaction executes a range query twice.

The second query returns **new or missing rows** because another transaction inserted/deleted matching rows.

Example:

```
SELECT *
FROM payment
WHERE amount > 1000;
```

Transaction A:

First query → 5 rows

Transaction B inserts another matching payment and commits.

Transaction A:

Second query → 6 rows

The extra row is a **phantom**.

10. Lost Update

This one is also important in interviews, although it is not one of the classic three anomalies used in the traditional isolation-level table.

Example:

Initial:

balance = 1000

Two transactions read it:

```
A reads 1000
B reads 1000
```

A calculates:

```
1000 - 100 = 900
```

B calculates:

```
1000 - 200 = 800
```

Then:

```
A writes 900  
B writes 800
```

A's update is effectively lost.

Final:

```
800
```

instead of the expected:

```
700
```

This is why **locking or optimistic concurrency control** can be required even when basic transaction support exists.

11. Isolation Level Comparison

The conventional interview table is:

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	✓ Possible	✓ Possible	✓ Possible
Read Committed	✗ Prevented	✓ Possible	✓ Possible
Repeatable Read	✗ Prevented	✗ Prevented	Depends on DB/implementation
Serializable	✗ Prevented	✗ Prevented	✗ Prevented

The important nuance is:

Exact behavior of REPEATABLE READ can differ by database implementation, especially when MVCC is involved.

So don't blindly claim that every database implements it exactly the same way.

12. READ UNCOMMITTED

Lowest isolation.

A transaction can potentially see uncommitted changes made by another transaction.

```
A modifies data
  ↓
B can potentially see it
  ↓
A rolls back
```

Problem:

Dirty reads

Advantage

High concurrency.

Disadvantage

Weak consistency guarantees.

Usually inappropriate for financial/payment data.

13. READ COMMITTED

A transaction only sees **committed data** from other transactions.

Example:

```
A updates balance = 500
A has not committed

B reads balance
→ B does not see 500
```

Once A commits, later reads by B can see the new value.

It prevents:

Dirty reads

But allows:

Non-repeatable reads
Phantom reads

This is a very common isolation level.

14. REPEATABLE READ

The goal is stronger read consistency.

Suppose:

A reads payment = PAID

Another transaction changes it to:

REFUNDED

A's subsequent read within the same transaction is designed not to unexpectedly change according to the isolation semantics of the database.

It prevents:

Dirty reads
Non-repeatable reads

But phantom behavior depends on the database's implementation and concurrency mechanism.

Many databases use **MVCC (Multi-Version Concurrency Control)** to provide snapshot-style reads.

15. SERIALIZABLE

Strongest standard isolation level.

The database behaves conceptually as though transactions execute **one after another**, even when the system internally uses concurrency.

Imagine:

Transaction A
Transaction B

Instead of allowing conflicting concurrent effects:

A
B

the database enforces behavior equivalent to:

A → B
or
B → A

This prevents:

- Dirty reads
- Non-repeatable reads
- Phantom reads

Cost

Potentially:

- more locking/contention
- more blocking

- lower concurrency
- more transaction retries/failures depending on implementation

So:

SERIALIZABLE gives stronger correctness at the cost of concurrency.

16. Simple Example

Suppose we have:

```
CREATE TABLE account (  
  id BIGINT PRIMARY KEY,  
  balance DECIMAL(10,2)  
);
```

Two transactions simultaneously withdraw money.

Transaction A

```
BEGIN;  
  
SELECT balance  
FROM account  
WHERE id = 1;  
  
UPDATE account  
SET balance = balance - 500  
WHERE id = 1;  
  
COMMIT;
```

Transaction B

```
BEGIN;  
  
SELECT balance  
FROM account  
WHERE id = 1;  
  
UPDATE account  
SET balance = balance - 700  
WHERE id = 1;  
  
COMMIT;
```

Now concurrency becomes important.

You need to consider:

Can both transactions read the same old balance?
Can one overwrite the other's update?
Can the account become negative?

This is where isolation, locking, and concurrency-control techniques become important.

17. Transactions in Spring Boot

In Spring, the most common way to define a transaction boundary is:

```
@Service
public class PaymentService {

    @Transactional
    public void processPayment(Long paymentId) {

        paymentRepository.save(payment);

        invoiceRepository.markAsPaid(paymentId);

        historyRepository.save(history);

    }
}
```

Conceptually:

```
@Transactional
↓
BEGIN TRANSACTION
↓
save payment
↓
update invoice
↓
save history
↓
COMMIT
```

If an appropriate exception causes rollback:

```
payment saved      ✓
invoice updated    ✓
history save       ✗
↓
ROLLBACK
↓
Earlier DB changes undone
```

18. Setting Isolation Level in Spring

Spring allows:

```
@Transactional(
    isolation = Isolation.READ_COMMITTED
)
public void processPayment() {
    ...
}
```

Other options include:

```
Isolation.READ_UNCOMMITTED
```

```
Isolation.READ_COMMITTED
Isolation.REPEATABLE_READ
Isolation.SERIALIZABLE
```

Example:

```
@Transactional(
    isolation = Isolation.REPEATABLE_READ
)
public void processOrder() {
    ...
}
```

Important

`@Transactional` does **not magically make everything transactional**.

It manages transactions through Spring's transaction infrastructure, typically around database operations handled by a transaction manager.

19. `@Transactional` : What Interviewers Often Ask

Does `@Transactional` create a new transaction every time?

Not necessarily.

By default, propagation is:

```
REQUIRED
```

Meaning:

```
Existing transaction?
↓
Yes → join it
↓
No → create one
```

This is transaction **propagation**, which is separate from isolation.

20. Isolation vs Propagation

A common interview trap.

Isolation

Answers:

How should concurrent transactions interact with each other?

Examples:

```
READ_COMMITTED
REPEATABLE_READ
SERIALIZABLE
```

Propagation

Answers:

What should happen when one transactional method calls another transactional method?

Examples:

```
REQUIRED
REQUIRES_NEW
NESTED
```

So:

```
Isolation → concurrent transaction behavior
Propagation → transaction boundary behavior
```

21. Isolation vs Locking

Another important distinction.

Isolation

A database-level concurrency guarantee.

Locking

A mechanism used to control concurrent access.

For example:

```
SELECT *
FROM account
WHERE id = 1
FOR UPDATE;
```

This can lock the selected row so another transaction cannot freely modify it until the transaction completes, depending on the database and isolation semantics.

In JPA you may see:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findById(Long id);
```

This is **peessimistic locking**.

Another approach is:

Optimistic locking

using a version field:

```
@Version  
private Long version;
```

22. Practical Payment Example

Suppose a payment request does:

1. Check invoice status
2. Create payment
3. Update invoice
4. Save payment history

A transaction may protect the database changes:

```
BEGIN  
|  
| — create payment  
| — update invoice  
| — save history  
|  
COMMIT
```

But there is an important distinction:

```
Database transaction  
  ≠  
External API transaction
```

For example, imagine:

```
BPS database transaction  
  ↓  
Call external payment provider  
  ↓  
Provider processes payment  
  ↓  
DB update fails
```

Your DB rollback **cannot automatically roll back the external provider's payment.**

That is one reason distributed systems use mechanisms such as:

- idempotency
- compensation
- reconciliation
- Saga
- transactional outbox
- retry handling

This is especially important for payment systems.

23. Interview-Ready Answer

A good 30–45 second answer:

A transaction is a logical unit of database work that should be completed atomically. ACID stands for Atomicity, Consistency, Isolation, and Durability. Atomicity ensures all operations succeed or roll back together. Consistency ensures database constraints and business invariants are preserved. Isolation controls how concurrent transactions interact and prevents problems such as dirty reads and non-repeatable reads. Durability ensures committed data survives failures. The standard isolation levels are Read Uncommitted, Read Committed, Repeatable Read, and Serializable. Higher isolation provides stronger consistency but can reduce concurrency and increase contention. In Spring Boot, transaction boundaries are commonly managed using `@Transactional` .

24. Common Interview Questions

Basic

1. What is a transaction?
2. Explain ACID.
3. Explain atomicity with an example.
4. What is consistency?
5. What is durability?
6. What is isolation?

Isolation

7. What is a dirty read?
8. What is a non-repeatable read?
9. What is a phantom read?
10. What is lost update?
11. Explain all four isolation levels.
12. Which isolation level prevents dirty reads?
13. Which isolation level provides the strongest isolation?
14. Why don't we always use SERIALIZABLE?
15. What is the difference between REPEATABLE READ and SERIALIZABLE?
16. Does REPEATABLE READ always prevent phantom reads?

Spring

17. How does `@Transactional` work?
18. What happens when a `@Transactional` method throws an exception?
19. What is transaction propagation?
20. Difference between `REQUIRED` and `REQUIRES_NEW` .
21. What is transaction isolation vs propagation?

22. What is optimistic locking?
23. What is pessimistic locking?
24. What does `@Version` do in JPA?
25. Can a database transaction roll back an external REST API call?

25. BPS-Specific Questions — Inspect Your Codebase

Don't assume the answers; use these to inspect your implementation.

Transaction boundary

- In BPS, where is `@Transactional` used?
- Is the transaction boundary at the **controller, service, or repository** layer?
- Which operations are intentionally grouped into one transaction?

Payment processing

- During payment processing, which DB operations must succeed or fail together?
- If payment creation succeeds but history insertion fails, what happens?
- Is rollback actually expected for that failure?

Isolation

- What isolation level is configured for BPS transactions?
- Is the database using the default isolation level?
- Is there any reason your payment/billing flow requires something stronger than `READ_COMMITTED` ?

Concurrency

- Can two requests process the same payment/invoice simultaneously?
- How does BPS prevent duplicate processing?
- Do you see `@Version` , `FOR UPDATE` , `PESSIMISTIC_WRITE` , unique constraints, or another concurrency mechanism?

External systems

- If BPS updates its DB and then calls Hydra/Chase, what happens if the external call succeeds but the DB transaction rolls back?
- How does BPS handle retries, duplicate requests, or reconciliation in that situation?

One mental model to remember

```
Transaction
├── Atomicity    → all or nothing
├── Consistency  → valid state → valid state
├── Isolation    → control concurrent interactions
└── Durability   → committed data survives

Isolation levels
```

- |
- |— READ UNCOMMITTED → weakest
- |— READ COMMITTED
- |— REPEATABLE READ
- |— SERIALIZABLE → strongest

The natural next topic is **optimistic vs pessimistic locking** + **@Version** , because interviewers often connect it directly to isolation and concurrent payment processing.